

Informatics II Exam FS 2024

February 17, 2025

Q1: Single Choice [8 points]

Consider the following C function X:

```
1 void X(int numbers[], int n) {
2     if (n <= 1) {
3         printf("NULL");
4         return;
5     }
6     int count = 1;
7     int num = numbers[n - 1];
8     for (int i = n - 2; i >= 0 ; i--) {
9         if (num == numbers[i]) {
10            count++;
11        } else {
12            printf("%d%d", num, count);
13            count = 1;
14            num = numbers[i];
15        }
16    }
17    printf("%d%d", num, count);
18 }
```

Assume array `numbers` and integer `n` have been declared and instantiated as follows:

```
1 int numbers[] = {1,1,2,2,2,3,2,2,1,1,1};
2 int n = 11;
```

What does the call `X(numbers, n)` print?

- ☐ 1322132312
- ☐ 1322312212
- ☐ 1222312313
- ☒ 1322312312
- ☐ NULL

Q2: Single choice [5 points]

Consider the following C function F:

```
1 void F(int i) {  
2     if (i <= 0) {  
3         printf("!");  
4     } else if (i == 1) {  
5         printf("*");  
6     } else {  
7         F(i - 2);  
8         F(i - 1);  
9     }  
10 }
```

What does the call F (5) print? What is the asymptotic time complexity of F (n) ?

- | | | |
|-------------------------------------|---------------------------|--|
| ☒ | F (5) prints
!!***!* | Asymptotic time complexity of F (n)
$\Omega(2^{n/3})$ |
| ☐ | F (5) prints
!!!**!* | Asymptotic time complexity of F (n)
$\Theta(n^2)$ |
| ☐ | F (5) prints
!!***!* | Asymptotic time complexity of F (n)
$\Theta(n^2)$ |
| ☐ | F (5) prints
!!!**!* | Asymptotic time complexity of F (n)
$\Omega(2^{n/3})$ |
| ☐ | F (5) prints
!!***!* | Asymptotic time complexity of F (n)
$\Theta(n^2)$ |

Q3: Single Choice [3 points]

What is the asymptotic time complexity for the recurrence $T(n) = 3 * T(n/3) + n$ if $T(1) = 1$?

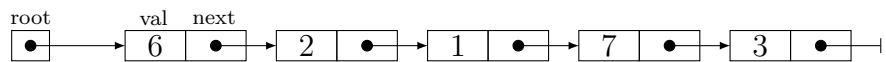
- ☐ $O(\log_3 n)$
- ☒ $O(n * \log_3 n)$
- ☐ $O(n^2 * (\log_3 n)^2)$
- ☐ $O(n^2 * \log_3 n)$
- ☐ $O(3^n)$

Q4: Single Choice [3 points]

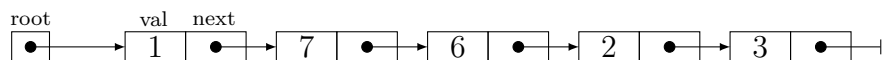
A linked list is defined as follows:

```
1 struct node {  
2     int val;  
3     struct node* next;  
4 };  
5  
6 struct node* root;
```

Assume a linked list has been instantiated to:



This list shall be transformed to the following list:

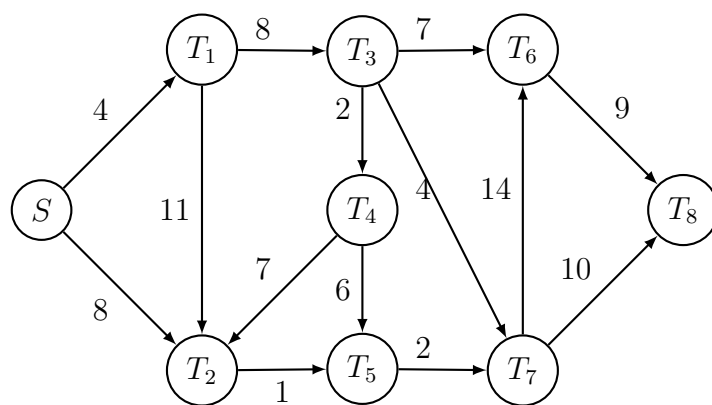


Assuming that only pointer values may be changed what is the minimum number of pointer value changes to do this transformation?

- ☐ 1
- ☐ 2
- ☒ 3
- ☐ 4
- ☐ 5

Q5: Single Choice [5 points]

Consider Dijkstra's **single-source shortest path** algorithm for the directed graph $G_3 = (V, E)$ with source vertex S . How often are node distances changed when relaxing edges during the execution of the algorithm?



$G_3 = (V, E)$

- ☐ 14
- ☐ 8
- ☒ 9
- ☐ 10
- ☐ 11

Q6: Open Text [8 points]

Consider a stack S that contains n unique integer values. Assume integers n and k with $n > 1$ and $1 < k \leq n$. Provide pseudocode for a function `Kth(S, n, k)` that returns the k -th largest integer of S . You are allowed to change S . You are allowed to use an auxiliary stack. Other composite auxiliary data structures may not be used.

The abstract data type `Stack` provides the following procedures and functions (in your solution no other procedures and functions may be used):

- `initStack()`: initializes and returns a stack
- `push(S, x)`: pushes value x onto stack S
- `pop(S)`: removes and returns the top value from a non-empty stack S
- `isEmpty(S)`: returns `True` if stack S is empty and `False` otherwise

Solution 1:

```
1 int Kth(struct Stack* S, int n, int k){
2     struct Stack* T = initStack();
3     int x;
4     for (int i = 1; i <= k; i++) {
5         int top = pop(S);
6         push(T, top);
7         x = top;
8         while (!isEmpty(S)) {
9             top = pop(S);
10            push(T, top);
11            if (top > x) {x = top;}
12        }
13        while (!isEmpty(T)) {
14            top = pop(T);
15            if (top != x) {push(S, top);}
16        }
17    }
18    free(T);
19    return x;
20 }
```


Solution 2:

```
1 int Kth(struct Stack* S, int n, int k){
2     struct Stack* T = initStack();
3     int x;
4     for (int i = 1; i <= k; i++) {
5         x = pop(S);
6         push(T, x);
7         while (!isEmpty(S)) {
8             int m = pop(T);
9             int n = pop(S);
10            if (m < n) {
11                push(T, m);
12                push(T, n);
13                x = n;
14            } else {
15                push(T, n);
16                push(T, m);
17                x = m;
18            }
19        }
20        pop(T);
21        while (!isEmpty(T)) {push(S, pop(T));}
22    }
23    free(T);
24    return x;
25 }
```

Solution 3:

```
1 int Kth(struct Stack* S, int n, int k){
2     struct Stack* T = initStack();
3     int i, j, x, y, z;
4     for (i = 1; i <= n-1; i++) {
5         for (j = 1; j <= n-1; j++) {
6             x = pop(S);
7             y = pop(S);
8             if (x < y) { z = x; x = y; y = z; }
9             push(S, y);
10            push(T, x);
11        }
12        while (!isEmpty(T)) {push(S, pop(T));}
13    }
14    free(T);
15    for (i = 1; i <= k; i++) { x = pop(S); }
16    return x;
17 }
```

Solution 4:

```
1 int Kth(struct Stack* S, int n, int k){
2     struct Stack* T = initStack();
3     push(T, pop(S));
4     while (isEmpty(S) == 0) {
5         int y = pop(S);
6         int count = 0; // the number of elements of T that are larger than top
7         while (isEmpty(T) == 0) {
8             int temp = pop(T);
9             if (temp > y) {
10                 push(S, temp);
11                 count ++;
12             } else {
13                 push(T, temp);
14                 break;
15             }
16         }
17         push(T, y); // insert top
18         while (count > 0) { // move back elements that are larger than top to T.
19             push(T, pop(S));
20             count --;
21         }
22     }
23     int x = pop(T);
24     for(int i = 1; i < k; i++) x = pop(T);
25     return x;
26 }
```

Q7: Single Choice [4 points]

Consider the function `WhatIDo(x, n)` with $1 \leq x \leq n$:

Algo: `WhatIDo(x, n)`

```
1 i = x;  
2 j = x;  
3 for (i = x; i <= n ; i++) do  
4   for (j = x; j <= n ; j = j * 2) do  
5     printf("Hello World");
```

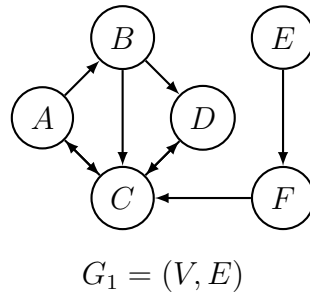
What are the best- and worst-case asymptotic time complexities for the call `WhatIDo(x, n)`, respectively?

- ☐ best case = $\Theta(1)$, worst case = $\Theta(n^2)$
- ☒ best case = $\Theta(1)$, worst case = $\Theta(n * \log n)$
- ☐ best case = $\Theta(n)$, worst case = $\Theta(n^2)$
- ☐ best case = $\Theta(n)$, worst case = $\Theta(n * \log n)$
- ☐ best case = $\Theta(1)$, worst case = $\Theta(n)$

Q8: Open Text [10 points]

Assume a directed graph $G = (V, E)$ and two vertices $v_1, v_2 \in V$.

Write an algorithm `FindAllPaths(v1, v2)` that prints **all simple paths** from v_1 to v_2 . A path is simple if no node is repeated.



For instance the output of algorithm `FindAllPaths(A, D)` on graph G_1 is

A B C D
A B D
A C D

since there are three simple paths (ABCD, ABD and ACD) from A to D. The order in which the paths are output does not matter.

Solution 1:

Modify BFS. Instead of maintaining a queue with *nodes* that have not yet been expanded we maintain a queue with *simple paths* that have not yet been expanded. We extend simple paths by going from the last node in a simple path to all neighbors that are not part of the path. We stop a path expansion and output a solution if the last node is v_2 .

Algo: FindAllPaths(v_1, v_2)

```
1 Q = [[v1]];
2 while Q != Empty do
3   L = Dequeue(Q);
4   h = head(L);
5   if h == v2 then print(L);
6   else
7     for u ∈ h.adj do
8       if u ∉ L then
9         Enqueue(Q, u ∘ L)
```

- Q = [[A]]
- Q = [[BA],[CA]]
- Q = [[CA],[CBA],[DBA]]
- Q = [[CBA],[DBA],[DCA]]
- Q = [[DBA],[DCA],[DCBA]]
- Q = [[DCA],[DCBA]]
- Q = [[DCBA]]
- Q = []

Solution 2:

Modify DFS. The first modification (FAP) makes sure that instead of visiting all nodes and edges the search traverses all simple paths. This is achieved by setting the color to white instead of black at the end of the DFS call so that the search does not stop when we hit forward or cross edges. The second modification (FAPpp) adds a stack that keeps track of all nodes on a path and prints a path that ends at the target node.

Algo: FAPpp(a,b)	
1 a.col := gray;	• S = [A]
2 push(S,a);	• S = [AB]
3 if a = b then	• S = [ABC]
4 printS(S)	• S = [ABCD] (print)
5 else	• S = [ABC]
6 for u ∈ a.adj do	• S = [AB]
7 if u.col = white then	• S = [ABD] (print)
8 FApp(u,b)	• S = [AB]
9 pop(S);	• S = [AC]
10 a.col := white;	• S = [ACD] (print)

Q9: Kprim [8 points]

Consider a hash table with m slots and hash function $h(k) = k \% m$. We use open addressing with **linear probing and step size one**.

Assume the hash table has $m = 8$ slots and the elements 70, 998, 300, 20, 17, 6, 5, 19 are inserted into the hash table. The order in which the elements are inserted into the hash table is unknown.

Determine if, independent of the insertion order, each of the following 4 elements is always inserted into exactly the same slot (check True) or not (check False).

Key	True	False
998	☐	☒
17	☐	☒
5	☐	☒
19	☒	☐

Q10: Text Question [9 points]

Consider an array A with n integers that are either zero, one or two. Write a C function `Sort3(int A[], int n)` that changes the order of elements in A such that zeros come first, ones come second, and twos come last. Your solution may use at most one loop and no composite auxiliary data structures are permitted.

```
void Sort3(int A[], int n) {
    int l = -1;
    int r = n;
    int i = 0;
    while (i < r) {
        if (A[i] == 0) {
            l++;
            A[i] = A[l];
            A[l] = 0;
            i++;
        } else if (A[i] == 2) {
            r--;
            A[i] = A[r];
            A[r] = 2;
        } else {
            i++;
        }
    }
}
```

Q11: Open Text [12 points]

Consider an unordered array A with n distinct integers ($n \geq 3$). Find three distinct numbers belonging to A whose product is maximum and return the maximum product P_3 .

Provide the code of a C function `int p3(int A[], int n)` that computes and returns P_3 of A . The time complexity of your solution may not be larger than $O(n)$ and no composite auxiliary data structures are permitted.

```
int p3(int A[], int n) {
    int P1 = 0, P2 = 0, P3 = 0, M1 = 0, M2 = 0;
    for (int i=0; i<n; i++) {
        if (A[i] > P1) { P3 = P2; P2 = P1; P1 = A[i]; }
        else if (A[i] > P2) { P3 = P2; P2 = A[i]; }
        else if (A[i] > P3) { P3 = A[i]; }
        if (A[i] < M1) { M2 = M1; M1 = A[i]; }
        else if (A[i] < M2) { M2 = A[i]; }
    }
    return P1 * max(P2 * P3, M1 * M2);
}
```

Q12: Open Text [15 points]

Consider an array A with n integers. Find a contiguous subarray $A[i..j]$ of A whose product is as large as possible. Your solution shall return the product of all numbers in $A[i..j]$.

Provide a C function `int psub(int A[], int n)` that computes and returns the product $P_i * \dots * P_j$. The time complexity of your solution may not be larger than $O(n)$.

dynamic programming:

data structure:

```
int dp1[N]; // dp1 = maximum product of subarray ending at "i"
int dp2[N]; // dp2 = minimum product of subarray ending at "i"
```

algorithm:

```
void psub(int A[]){
    dp1[0] = A[0];
    dp2[0] = A[0];
    for (int i = 1; A[i]!=NaN; i++) {
        dp2[i] = min(min(dp2[i-1]*A[i], dp1[i-1]*A[i]), A[i]);
        dp1[i] = max(max(dp1[i-1]*A[i], dp2[i-1]*A[i]), A[i]);
    }
}
```

call/use:

```
psub(C[k]);

int m = dp1[0];
for (int i=1; C[k][i]!=NaN; i++) { m = max(m,dp1[i]); }
printf("\nMaximum_product_of_a_subarray_is_%d", m);
```

memoization:

data structure:

```
int dp1[N]; // dp1 = maximum product of subarray ending at "i"
int dp2[N]; // dp2 = minimum product of subarray ending at "i"
```

algorithm:

```
int minCA(int A[], int i) {
    if (i == 0) { return A[0]; }
    if (dp2[i] != NaN) { return dp2[i]; }
    dp2[i] = min(min(maxCA(A, i-1)*A[i], minCA(A, i-1)*A[i]), A[i]);
    return dp2[i];
}

int maxCA(int A[], int i) {
    if (i == 0) { return A[0]; }
    if (dp1[i] != NaN) { return dp1[i]; }
    dp1[i] = max(max(minCA(A, i-1)*A[i], maxCA(A, i-1)*A[i]), A[i]);
    return dp1[i];
}
```

call/use:

```
for (int i=1; i<N; i++) { dp1[i] = NaN; dp2[i] = NaN; }

int l = maxCA(C[k], 0);
for (int i=1; C[k][i]!=NaN; i++) { l = max(l, maxCA(C[k], i)); }
printf("\nMaximum product of a subarray is %d", l);
```